

CELF: Cost-Effective Outbreak Detection in Networks

With Advanced Optimization Techniques

Abhishek Tiwari (02)

Isha Adhikari (19)

Abstract

In modern world communities networks Outbreak Detection in networks become a critical problem in network science. Outbreak detection in networks means to select a small subset of nodes in the network that can influence most number of nodes in the network. It helps to detect spreading processes means the most influential or powerful node that can influence or spread the information to maximum number of nodes such as epidemics, rumors, panic creation, communities riots or malware. It helps to detect that nodes and break that nodes as early as possible to stop that epidemic or rumor. The CELF (Cost-Effective Lazy Forward) algorithm [1] is an efficient greedy approach that can use for outbreak detection. It leverages sub-modularity to reduce computation cost [1]. This report presents a detailed study of the CELF algorithm, its working mechanism, time complexity, optimizations, advantages, and limitations. Furthermore, in this report we purpose some advanced optimizations and analyze them in detail. These optimizations includes Hybrid CELF (ML Heuristic approach), Edge Pruning CELF and Community-Aware CELF to improve efficiency, scalability and performance in large-scale networks.

1 Introduction

In modern world networked systems such as social networks, communication networks, criminal networks and biological systems networks, information and influence spread rapidly through interconnected nodes [5]. The rumors, epidemic and panic creation messages float fastly in to the communities so to stop them detect such outbreaks or most influential nodes early is crucial for applications and countries welfare.

However, monitoring or checking each and every node in a network is computationally expensive, impractical and also time consuming task. Therefore, the problem is to select a small subset of nodes in the network, known as sensors or seed set, that can effectively detect outbreaks or detect most influential nodes in the network.

2 CELF Algorithm

The CELF (Cost-Effective Lazy Forward) algorithm [1] is an optimized greedy algorithm designed to overcome the outbreak detection challenge also known as influence maximization problem efficiently [1]. It is used for outbreak detection which discussed earlier in detail.

2.1 Key Idea

CELF improves the classical greedy algorithm by exploiting the property of **submodularity**, which ensures diminishing returns [2].

2.2 Submodularity

A function $f(S)$ is submodular if:

$$f(S \cup \{v\}) - f(S) \geq f(T \cup \{v\}) - f(T)$$

for $S \subseteq T$. [1]

This property allows CELF to avoid recomputing marginal gains repeatedly.

3 Working of CELF

- Initialize an empty sensor set or seed set
- Compute marginal gain of all nodes
- Store nodes in a priority queue
- Select node with highest gain
- Recompute only when necessary (lazy evaluation)

4 Example

Consider a graph with nodes $\{1, 2, 3, 4, 5\}$.

Marginal Gain:

- $f(1) = 3$
- $f(2) = 4$
- $f(3) = 2$
- $f(4) = 3$
- $f(5) = 3$

Step 1: Select B
Step 2: Recompute gains
Step 3: Select A

Final sensors:

$$S = \{A, B\}$$

5 Algorithm

Initialize S = empty
Compute marginal gains
Store in priority queue

```
while  $|S| < k$ :  
    pick best node  
    if gain valid:  
        add to  $S$   
    else:  
        recompute  
return  $S$ 
```

6 Python Implementation

```
def outbreak_influence(G, S):  
    influenced = set(S)  
    for node in S:  
        influenced.update(G.neighbors(node))  
    return len(influenced)  
  
def CELF_network(G, k):  
    S = set()  
    heap = []  
    iteration = 0  
  
    for node in G.nodes():  
        gain = outbreak_influence(G, {node})  
        heapq.heappush(heap, (-gain, node, 0))  
  
    while len(S) < k:  
        neg_gain, node, last = heapq.heappop(heap)  
  
        if last == iteration:  
            S.add(node)  
            iteration += 1
```

```

    else:
        new_gain = outbreak_influence(G, S | {node}) \
                    - outbreak_influence(G, S)
        heapq.heappush(heap, (-new_gain, node, iteration))
return S

```

7 Complexity Analysis

Naive Greedy:

$$O(k \cdot |V| \cdot f)$$

CELf:

$$O(|V| \log |V| + k \cdot f)$$

Space Complexity:

$$O(|V|)$$

8 Limitations of CELf

- High computation cost for large networks
- Repeated influence calculations
- No structural awareness in networks
- Lack of fairness and diversification

9 Advanced Optimizations

9.1 1. Hybrid CELf

This optimization approach combines ML heuristic filtering with CELf [3].

Idea:

- Firstly rank nodes using degree and betweenness centrality
- Select top nodes in the given network
- Apply CELf on reduced graph

Advantage: Using this optimization it reduces computation while maintaining accuracy.

9.2 2. Edge Weighting and Pruning

Edges are assigned weights based on:

- Common neighbors
- Degree of the node

$$Weight(u, v) = \alpha \cdot CN + \beta \cdot Degree$$

Low-weight edges are removed and high weight edges kept.

Advantage:

- Reduces the graph size
- Improves efficiency

9.3 3. Community-Aware CELF

Networks are divided into communities using Louvain method [4].

Steps:

- Firstly detect communities using Louvain method[4]
- Allocate sensors or seed nodes proportionally for each communities
- Apply Hybrid CELF for each communities

Advantage:

- Better coverage in networks
- This optimization improves scalability and performance

10 Performance Evaluation

For evaluating our optimizations we compare our optimizations with other algorithms like Greedy approach for influence maximisation, CELF and CELF++ on different datasets. We use three datasets for comparison.

Metrics:

- Execution Time
- Coverage

Dataset 1 (D1): Barabasi Albert Graph (Nodes:5000, Edges:15000)

Dataset 2 (D2): Barabasi Albert Graph (Nodes:10000, Edges:99900)

Dataset 3 (D3): ca-GrQc dataset (Nodes:5500, Edges:15500)

Observation:

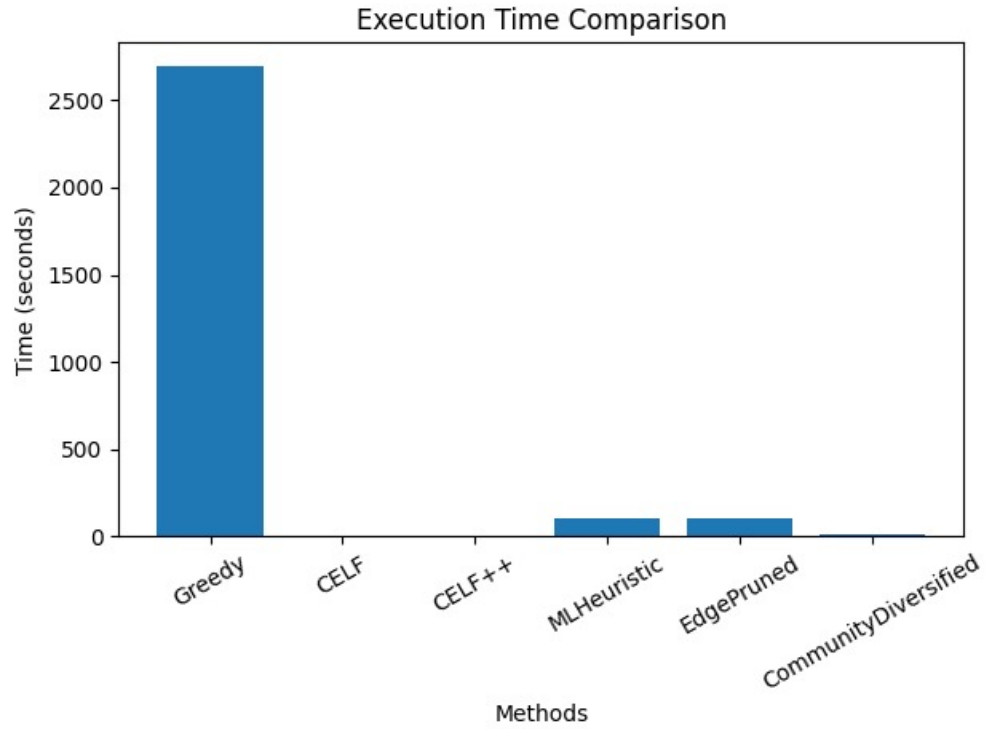


Figure 1: D1 Time Comparison

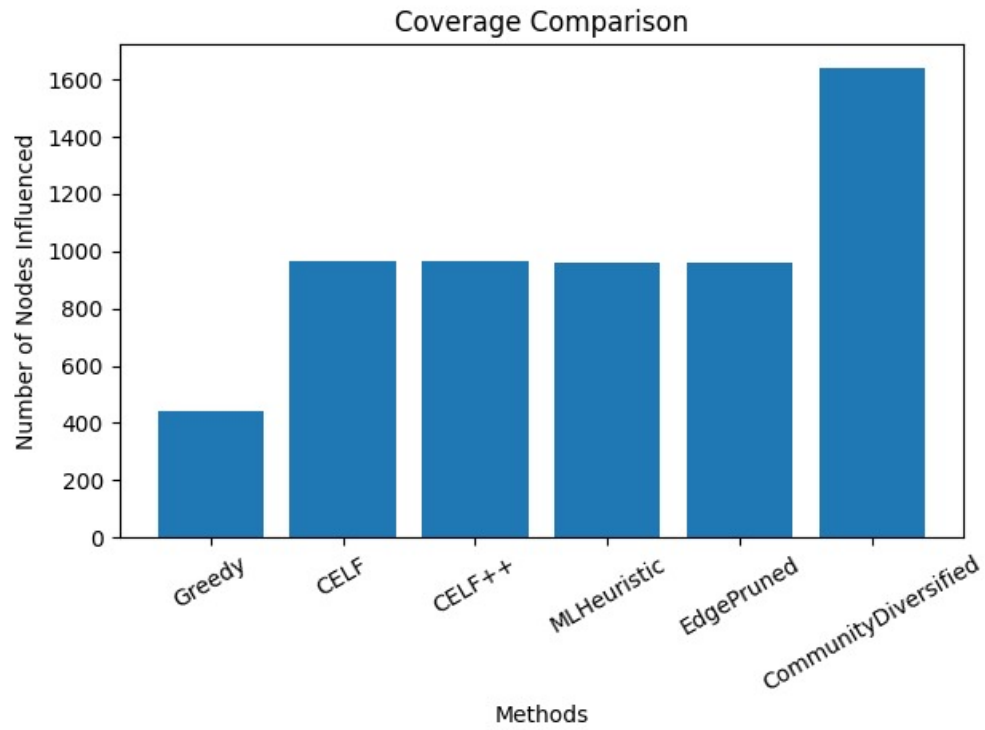


Figure 2: D1 Nodes Coverage Comparison

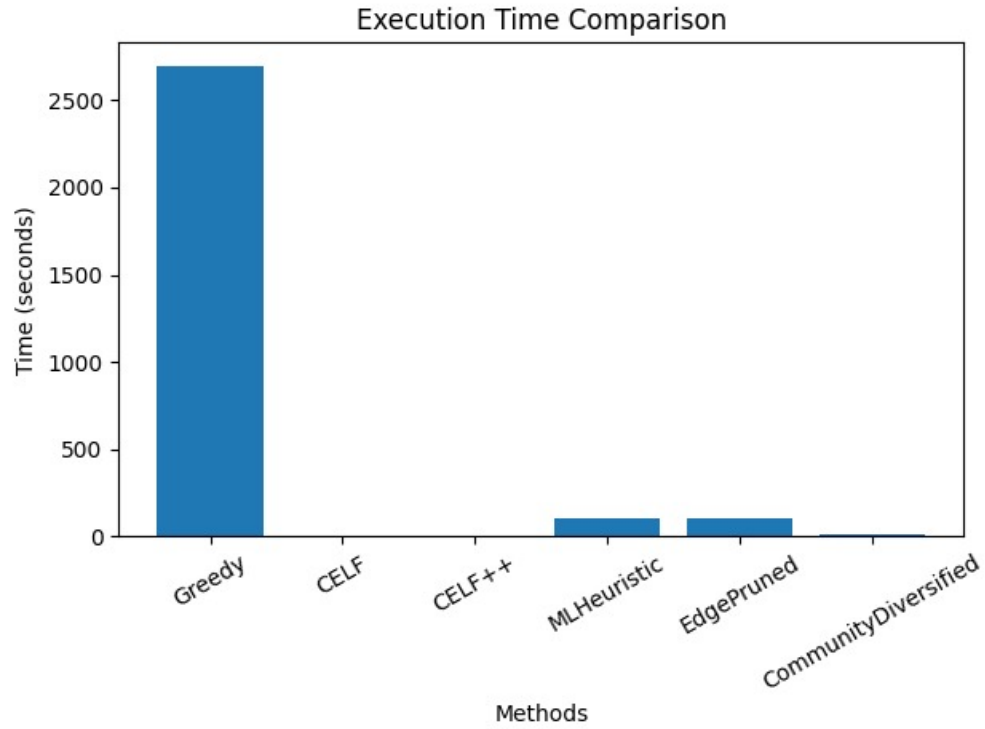


Figure 3: D2 Time Comparison

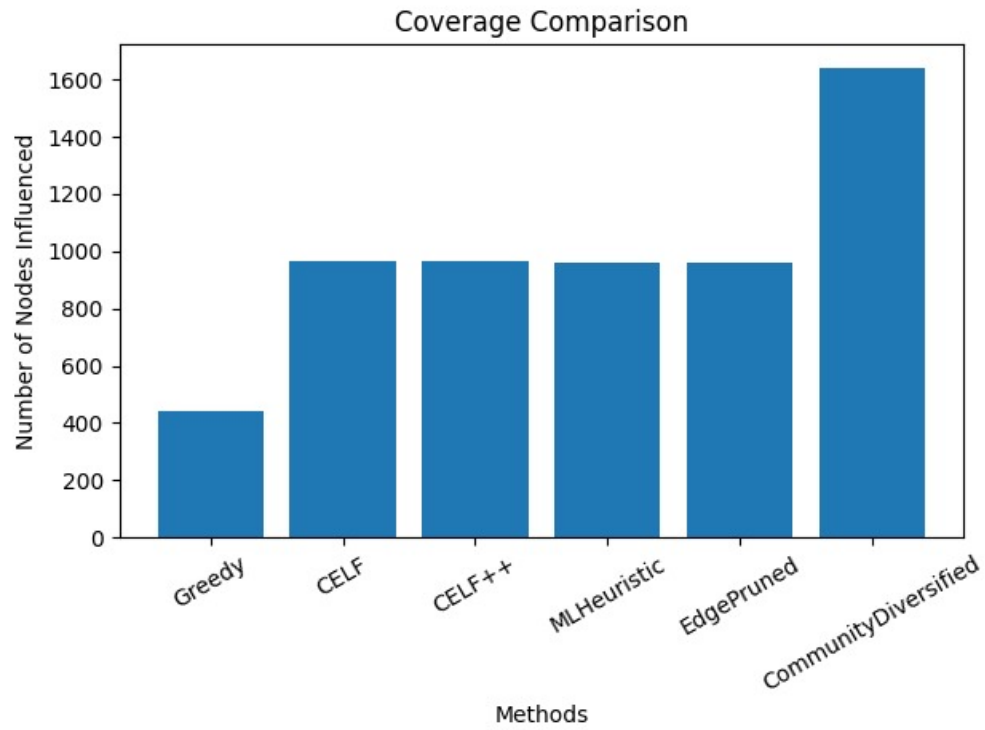


Figure 4: D2 Nodes Coverage Comparison

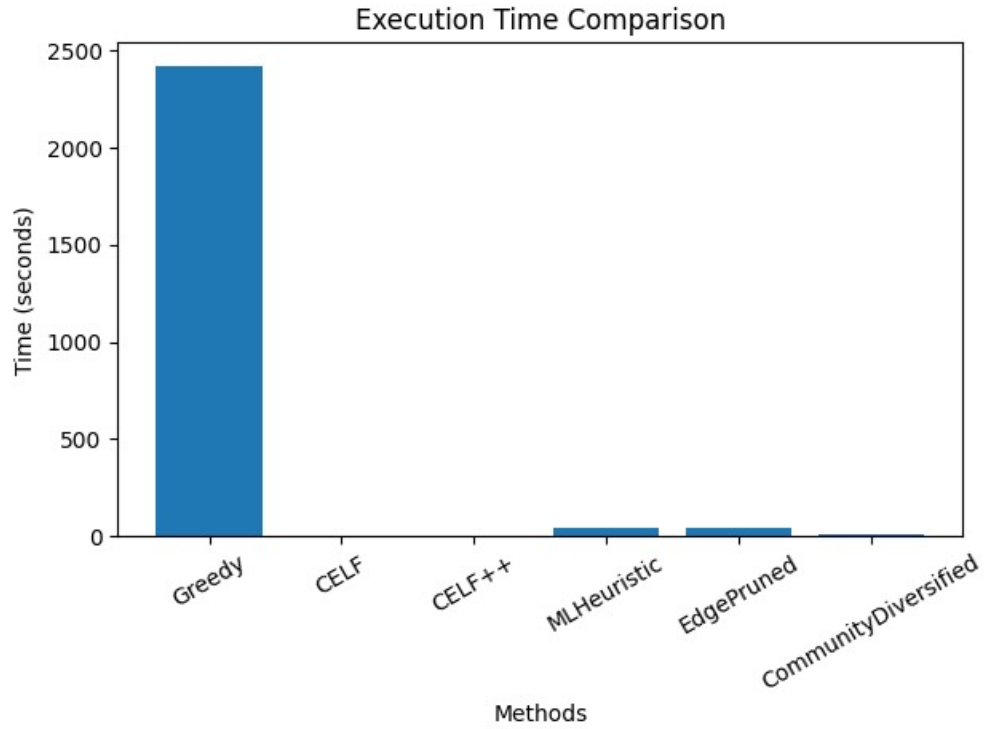


Figure 5: D3 Time Comparison

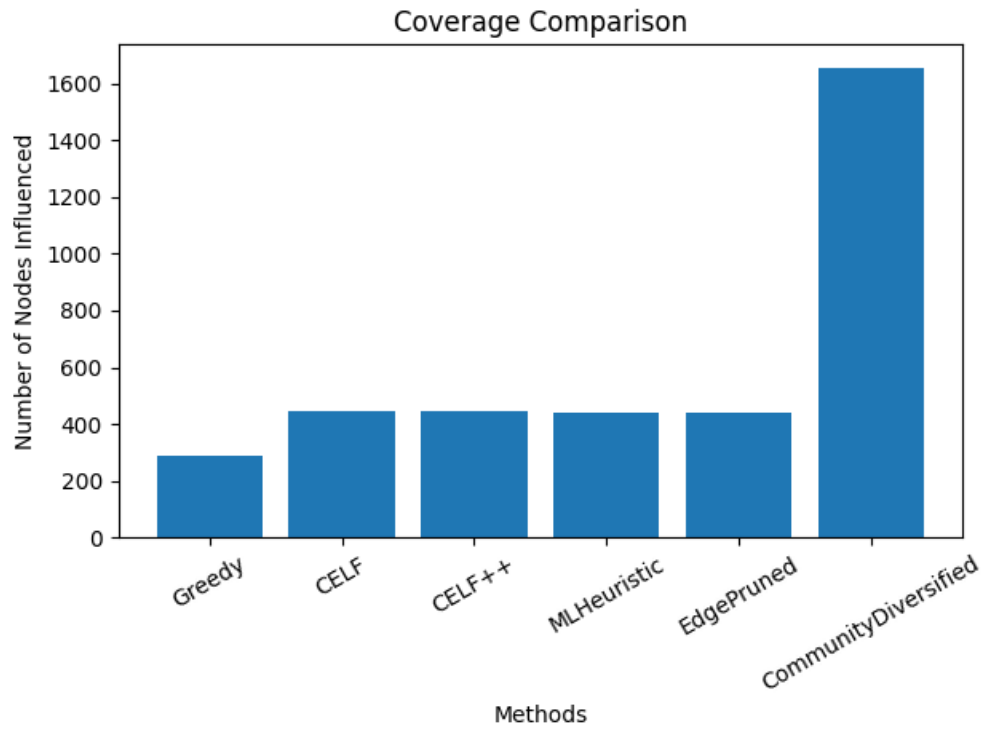


Figure 6: D3 Nodes Coverage Comparison

- CELF: Good accuracy, Low time
- Hybrid: Time Consuming, near-optimal
- Pruned: Time Consuming, slight loss
- Community: Best Balanced Performance with low time

11 Applications

- Epidemic and rumor detection in social networks
- Malware spread detection in criminal networks
- Social media monitoring or checking unlawful activities
- Uses in emergency alert systems to alert societies for such problems

12 Conclusion

CELF is an efficient algorithm for outbreak detection that improves greedy selection using lazy evaluation. However, it faces scalability and computational challenges in large networks. So to tackle that challenges we proposed optimizations significantly improve performance while maintaining high accuracy.

13 References

References

- [1] J. Leskovec, A. Krause, C. Guestrin, C. Faloutsos, J. VanBriesen, and N. Glance, “Cost-effective outbreak detection in networks,” *Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2007.
- [2] D. Kempe, J. Kleinberg, and É. Tardos, “Maximizing the spread of influence through a social network,” *Proceedings of the 9th ACM SIGKDD*, 2003.
- [3] W. Chen, Y. Wang, and S. Yang, “Scalable influence maximization for prevalent viral marketing in large-scale social networks,” *Proceedings of the 16th ACM SIGKDD*, 2009.
- [4] V. D. Blondel, J. L. Guillaume, R. Lambiotte, and E. Lefebvre, “Fast unfolding of communities in large networks,” *Journal of Statistical Mechanics*, 2008.
- [5] M. Newman, *Networks: An Introduction*, Oxford University Press, 2010.